arXiv:2012.12145v2 [math.OC] 28 Jan 2021

A COLREGs-Compliant Motion Planner for Autonomous Maneuvering of Marine Vessels in Complex Environments *

Kristoffer Bergman † Oskar Ljungqvist ‡ Jonas Linder § Daniel Axehill †

Abstract **—An enabling technology for future sea transports is safe and energy-efficient autonomous maritime navigation in narrow environments with other marine vessels present. This requires that the algorithm controlling the ship is able to account for the vessel's dynamics, and obeys the rules specified in the international regulations for preventing collision at sea (COLREGs). To account for these properties, this work proposes a two-step optimization-based and COLREGs-compliant motion planner tailored for large vessels. In the first step, a lattice-based motion planner is used to compute a suboptimal trajectory based on a library of precomputed motion primitives. To comply with the rules specified in COLREGs, the lattice-based planner is augmented with discrete states that represent what type of COLREGs situations that are active with respect to other nearby vessels. The trajectory computed by the lattice-based planner is then used as warm-start for the second receding-horizon step based on direct optimal control techniques. The aim of the second step is to solve the problem to local optimality with the discrete COLREGs states remained fixed. The applicability of the proposed motion planner is demonstrated in simulations, where it computes energy-efficient and COLREGs-compliant trajectories. The results also show that the motion planner is able to prevent complex collision situations from occurring.**

1 Introduction

The interest in autonomous ship technology has over the past decade rapidly increased. The main driving forces in the development of future sea transports are to improve efficiency, e.g., to minimize the environmental footprint by reducing fuel consumption, and to increase safety by reducing the number of accidents caused by human errors [Wingrove, 2020, International Maritime Organization, 2020]. An enabling technology to achieve these goals is autonomous maritime navigation in narrow environments such as archipelagos, when there are other manned and unmanned marine vessels present. As a consequence, it is important that the algorithm controlling the ship is able to avoid collisions with both static and dynamic obstacles[1]. In maritime navigation, the rules specified by the international regulations for preventing collisions at sea (COLREGs) must be followed when there is a risk of collision between vessels [Cockcroft and Lameijer, 2003]. Hence, it is required that algorithms in the control system comply with these rules.

In the literature, there exist a number of works that have focused on collision avoidance algorithms which comply with COLREGs. In [Kuwata et al., 2013], a motion planner is developed based on so-called velocity obstacles. The same concept is used in [Kufoalor et al., 2018], where it is extended to include cooperative behavior. Another COLREGs-compliant motion planner is presented in [Hagen et al., 2018], where a receding-horizon approach is used based on input sampling to a closed-loop controller. In [Eriksen et al., 2020], a hierarchical approach for collision avoidance is proposed, where a trajectory is computed in a first step using optimal control without considering dynamic obstacles. In a second step, a receding horizon controller is used to track the nominal trajectory while complying with COLREGs. Common for [Kuwata et al., 2013, Kufoalor et al., 2018, Hagen et al., 2018, Eriksen et al., 2020] are that they all use some sort of nominal reference that is computed without considering dynamic obstacles.

A probabilistic approach is to use the closed-loop variant of the rapidly-exploring random tree (RRT) algorithm for COLREGs-compliant motion planning [Chiang and Tapia, 2018], where the input to an underlying controller is sampled to generate feasible trajectory candidates. This planner is shown to efficiently compute COLREGs-compliant trajectories in several scenarios but is unable to optimize the trajectories with respect to a performance measure.

A method based on a COLREGs-compliant repairing A* graph search is proposed in [Campbell et al., 2014]. The algorithm first computes a path while neglecting the system dynamics, and then smoothen the path using splines. This method works well for agile unmanned surface vehicles (USVs). However, to be able to compute safe and energy-efficient trajectories for large vessels, a motion planner that is able to account for the vessel's dynamics is required. There are many papers that propose optimization-based motion planners for computing safe and energy-efficient trajectories for vessels, e.g., [Martinsen et al., 2019, 2020, Bitar et al.,

*This work was partially supported by FFI/VINNOVA and the Wallenberg Artificial Intelligence, Autonomous Systems and Software Program (WASP) funded by Knut and Alice Wallenberg Foundation; and Saab AB.

†Kristoffer Bergman and Daniel Axehill are with the Division of Automatic Control, Linköping University, Linköping, Sweden (e-mail: {kristoffer.bergman, daniel.axehill}@liu.se)

‡Oskar Ljungqvist is at Volvo Trucks, Gothenburg, Sweden, e-mail: oskar.ljungqvist@volvo.com.

§Jonas Linder is at ABB AB, Corporate Research, Västerås, Sweden (e-mail: jonas.x.linder@se.abb.com)

[1]Further on in this paper, the term "ship" will be used to denote the ship that is being controlled, and "dynamic obstacles" to denote other marine vessels covered by COLREGs.

2020]. However, neither of these works consider COLREGs.

This work proposes a two-step optimization-based motion planner for marine vessels that complies with COLREGs. It is based on the two-step algorithm presented in [Bergman et al., 2020b] and is here extended to account for dynamic obstacles while complying with COLREGs. In the first step, a lattice-based motion planner [Pivtoraiko et al., 2009] is used to compute a suboptimal trajectory to a discretized version of the motion planning problem. By employing classical graph-search algorithms, the lattice-based planner concatenates motion segments from a finite library of optimized *motion primitives* [Bergman et al., June 2019]. This step is here augmented with discrete states that represent what types of COLREGs situations that are active for the surrounding dynamic obstacles in each time step. To improve the trajectory computed by the lattice-based planner, a second receding-horizon step based on direct optimal control techniques is added that solves the problem to local optimality while keeping the discrete COLREGs states fixed. Together, the two steps constitute the proposed COLREGs-compliant motion planning algorithm. In an online setting, an update in the predicted trajectories of the dynamic obstacles is allowed to trigger re-planning.

The proposed framework allows to use a high-fidelity *maneuvering* model of the vessel [Fossen, 2011], making it possible to compute safe and energy-efficient trajectories in narrow environments. Compared to other approaches for COLREGs-compliant motion planning that modifies a trajectory based on a nominal reference, the proposed method uses an economic approach and computes the nominal trajectory based only on the initial and terminal states and the objective function. This will potentially prevent complex collision situations from occurring at all.

The remainder of the paper is organized as follows. In Section 2 the dynamical model of the ship is briefly described. In Section 3, the COLREGs-compliant motion planning problem is presented, and the proposed framework for solving the problem is given in Section 4. A simulation study is presented in Section 5 and the paper is concluded in Section 6 with a summary of the presented work as well as some suggestions for potential future research.

2 Ship modeling

This section contains a brief description of the dynamic ship model used for motion planning. The ship is assumed to move on the ocean surface and it is thus sufficient to consider the horizontal 3 degrees of freedom motion. The motion of the ship is described using two coordinate systems: an Earth-fixed (inertial) system and a body-fixed system that is located at the center of mass of the ship. The Earth-fixed generalized position is given by $\boldsymbol{\eta} = \begin{bmatrix} x & y & \psi \end{bmatrix}^\intercal \in \mathbf{SE}(2)$ [LaValle, 2006] and the body-fixed generalized velocity vector is represented by $\boldsymbol{\nu} = \begin{bmatrix} u & v & r \end{bmatrix}^\intercal \in \mathbf{R}^3$, see Fig. 1. The generalized velocity and position is related through the kinematics

$$\dot{\boldsymbol{\eta}} = \boldsymbol{R}(\psi)\boldsymbol{\nu}. \tag{1}$$

where $\boldsymbol{R}(\psi)$ is the rotation matrix. The motion induced by forces acting on the ship is derived using rigid-body mechanics and theory of hydrodynamics, see [Fossen, 2011] for details. The model is given by

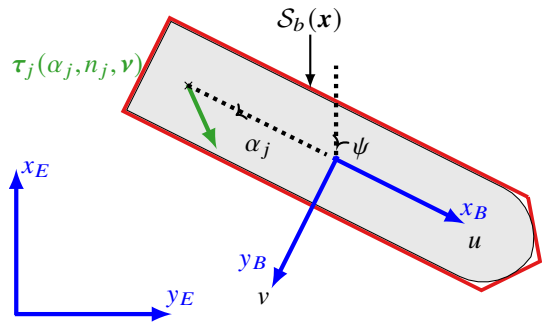


Figure 1: Definition of Earth-fixed (x_E, y_E) and body-fixed (x_B, y_B) coordinate systems, body-fixed velocities (u, v) and the convex polytope $\mathcal{S}_b(\boldsymbol{x})$ representing the bounding region of the ship's body at $\boldsymbol{x}$. Furthermore, the force vector of the jth thruster is illustrated.

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \boldsymbol{D}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau}, \tag{2}$$

where $\boldsymbol{M}$ is the total inertia matrix, $\boldsymbol{C}(\boldsymbol{\nu})\boldsymbol{\nu}$ corresponds to Coriolis and centripetal forces, $\boldsymbol{D}(\boldsymbol{\nu})\boldsymbol{\nu}$ describes the damping and $\boldsymbol{\tau}$ represents the forces that are acting on the ship. Simplified models for the thrusters are used where it is assumed that forces induced by the propellers and rudders can be separated. The total forces can be written as

$$\boldsymbol{\tau}(\boldsymbol{\alpha}, \boldsymbol{n}, \boldsymbol{\nu}) = \sum_{j=1}^{N_t} \boldsymbol{T}_j \boldsymbol{\tau}_j(\alpha_j, n_j, \boldsymbol{\nu}), \tag{3}$$

where N_t is the number of thrusters, $\boldsymbol{T}_j$ describes the geometry of the thruster configuration for thruster j and the vectors $\boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 & \dots & \alpha_{N_t} \end{bmatrix}^\intercal$ and $\boldsymbol{n} = \begin{bmatrix} n_1 & \dots & n_{N_t} \end{bmatrix}^\intercal$ represent the thruster angles and velocities. For more details regarding thruster and ship modeling, the reader is referred to, e.g., [Bergman et al., 2020b, Fossen, 2011, Perez, 2006].

To account for dynamics in the thrusters, the derivatives of the thruster angles and velocities are considered as control inputs to the system. A compact notation of the state vector is thus defined as $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{\eta}^\intercal & \boldsymbol{\nu}^\intercal & \boldsymbol{\alpha}^\intercal & \boldsymbol{n}^\intercal \end{bmatrix}^\intercal$ and the control-input vector is $\boldsymbol{u} = \begin{bmatrix} \dot{\boldsymbol{\alpha}}^\intercal & \dot{\boldsymbol{n}}^\intercal \end{bmatrix}^\intercal$. Then by combining (1)-(3), the ship dynamics can be written as

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}). \tag{4}$$

The feasible set for the state space is given by

$$\mathcal{X} = \{\boldsymbol{x} \mid ||\begin{bmatrix} u & v \end{bmatrix}|| \leq v_{\max},\ |n_j| \leq \bar{n}, \quad j = 1, \dots, N_t\}, \tag{5}$$

where $v_{\max}$ is the maximum allowed magnitude of the ship's velocity and $\bar{n}$ is the maximum speed for the thrusters. The control inputs are restricted to

$$\mathcal{U} = \{\boldsymbol{u} \mid |\dot{\alpha}_j| \leq \bar{\dot{\alpha}}, \quad |\dot{n}_j| \leq \bar{\dot{n}}, \quad j = 1, \dots, N_t\}, \tag{6}$$

where $\bar{\dot{\alpha}}$ and $\bar{\dot{n}}$ are the maximum angular rate and acceleration of the thrusters, respectively.

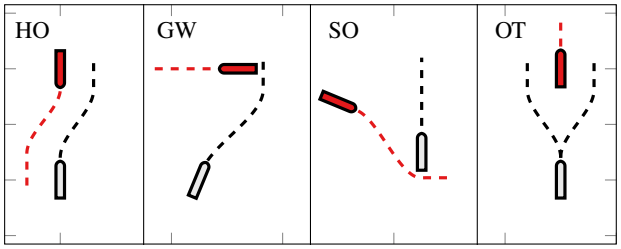


Figure 2: The main COLREGs scenarios together with the conceptually correct vessel behaviors. The ship is grey and the obstacle vessel is red. From left: Head on (HO), give way (GW), stand on (SO) and overtaking (OT).

The ship is assumed to operate in an environment with both static and dynamic obstacles. The bounding region of the ship's body at state $\boldsymbol{x}$ is defined by $\mathcal{S}_b(\boldsymbol{x})$, see Fig. 1. The region occupied by static obstacles is given by $\mathcal{X}^s_{\text{obst}}$, and the time-varying region for dynamic obstacles is $\mathcal{X}^d_{\text{obst}}(t)$. This region can, e.g., be described as the union of convex polytopes which depend on the states of the dynamic obstacles at time t. Thus, it is possible to pose the complete obstacle region as the union of static and dynamic obstacles

$$\mathcal{X}_{\text{obst}}(t) = \mathcal{X}^s_{\text{obst}} \cup \mathcal{X}^d_{\text{obst}}(t). \tag{7}$$

The free space where the vessel is not in collision with any obstacle is defined as $\mathcal{X}_{\text{free}}(t) = \mathcal{X} \setminus \mathcal{X}_{\text{obst}}(t)$. Since this set is the complement set of $\mathcal{X}_{\text{obst}}(t)$, it is in general not convex.

3 COLREGs-compliant motion planning

This section contains the description of a mathematical representation of the COLREGs regulations, which is used to formulate the COLREGs-compliant motion planning problem.

3.1 COLREGs representation

The regulations covered by COLREGs are divided into several parts and there are in total 41 rules. For the purpose of planning trajectories for collision avoidance, part B is most relevant. The rules covered in this work are Rule 8 and Rule 13-17, see [Cockcroft and Lameijer, 2003] for details. The corresponding COLREGs situations that are handled are

- **Safe (SF)**. The vessels are far away or heading away from each other.
- **Head on (HO)**. The vessels are meeting on a near reciprocal course with a risk of collision.
- **Give way (GW)**. Crossing where the dynamic obstacle approaches from the starboard side.
- **Stand on (SO)**. Crossing where the dynamic obstacle approaches from the port side, or when being overtaken.
- **Overtaking (OT)**. The ship is about to overtake the dynamic obstacle.

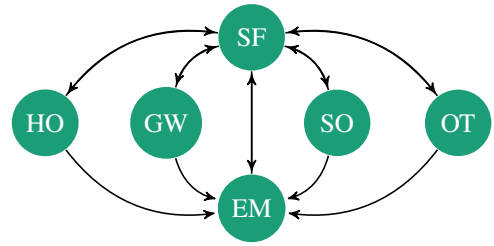


Figure 3: Allowable switches between COLREGs states.

- **Emergency (EM)**. When the dynamic obstacle is not complying with COLREGs.

Some of these situations are illustrated in Fig. 2. A discrete state $q^i(t) \in \mathcal{Q}$ is used to represent which of these situations that is active for the ship with respect to dynamic obstacle i, where $\mathcal{Q} = \{\text{SF}, \text{HO}, \text{OT}, \text{GW}, \text{SO}, \text{EM}\}$.

Each dynamic obstacle is modeled using a finite-state machine. The allowable switches between states in $\mathcal{Q}$ are illustrated in Fig. 3. Common choices for switching criterias are to use measures based on closest point of approach (CPA) [Kuwata et al., 2013, Eriksen et al., 2020, Kufoalor et al., 2018]. The time to CPA is calculated by assuming that the ship and dynamic obstacle keep their current velocity $\boldsymbol{v} = \begin{bmatrix} u & v \end{bmatrix}^\intercal$ constant. Define $\boldsymbol{p} = \begin{bmatrix} x & y \end{bmatrix}^\intercal$ and $\boldsymbol{v}$ as the position and velocity of the ship, and $\boldsymbol{p}_o$ and $\boldsymbol{v}_o$ as the position and velocity of the dynamic obstacle. Then, it is possible to calculate the time to CPA according to

$$t_{\text{CPA}} = \begin{cases} 0 & \|\boldsymbol{v} - \boldsymbol{v}_o\| \leq \epsilon, \\ \frac{(\boldsymbol{p}_o - \boldsymbol{p}) \cdot (\boldsymbol{v} - \boldsymbol{v}_o)}{\|\boldsymbol{v} - \boldsymbol{v}_o\|^2} & \text{otherwise.} \end{cases} \tag{8}$$

This expression is derived by solving the equation $\frac{\delta}{\delta t}\|(\boldsymbol{p} + t\boldsymbol{v}) - (\boldsymbol{p}_o + t\boldsymbol{v}_o)\| = 0$ with respect to t [Kufoalor et al., 2018]. The distance between the ship and the dynamic obstacle at CPA is

$$d_{\text{CPA}} = \|(\boldsymbol{p} + t_{\text{CPA}}\boldsymbol{v}) - (\boldsymbol{p}_o + t_{\text{CPA}}\boldsymbol{v}_o)\|. \tag{9}$$

Similar to what is done in [Eriksen et al., 2020], a switch from state SF to either HO, GW, SO or OT (see Fig 3) occurs when

$$(d_{\text{CPA}} \leq \underline{d}) \wedge (\underline{t}_{\text{low}} \leq t_{\text{CPA}} \leq \underline{t}_{\text{high}}), \tag{10}$$

where $\underline{d}$, $\underline{t}_{\text{low}}$, $\underline{t}_{\text{high}}$ are threshold parameters which can be state dependent. If (10) is true, a switch to a new COLREGs state occurs. As in [Tam and Bucknall, 2010], the COLREGs state is selected based on the relative position and heading of the dynamic obstacle with respect to the ship. A switch back to state SF occurs when

$$(d_{\text{CPA}} \leq \bar{d}) \vee \neg(\bar{t}_{\text{low}} \leq t_{\text{CPA}} \leq \bar{t}_{\text{high}}), \tag{11}$$

where $\underline{d} < \bar{d}$, $\bar{t}_{\text{low}} < \underline{t}_{\text{low}}$ and $\underline{t}_{\text{high}} < \bar{t}_{\text{high}}$ are threshold parameters that implement hysteresis and thus avoid fast switches between states. Note that it is not possible to switch between any of the situations described in Fig. 2 without first switching to SF. This is according to what is described in the regulations [Cockcroft and Lameijer, 2003].

A switch to EM only occurs during re-planning if any of the surrounding dynamic obstacles do not comply with COLREGs, or if a new dynamic obstacle is detected that is dangerously close to the ship. As in [Eriksen et al., 2020], a critical distance d_{crit} is defined to handle switches to EM. The time to the critical point t_{crit} is found by solving

$$\|(\boldsymbol{p}+t_{\text{crit}}\boldsymbol{v})-(\boldsymbol{p}_o+t_{\text{crit}}\boldsymbol{v}_o)\| = d_{\text{crit}}. \tag{12}$$

If $t_{\text{crit}} < \underline{t}_{\text{crit}}$, where $\underline{t}_{\text{crit}}$ is a user-defined threshold parameter, the state is switched to EM. Note in Fig. 3 that it is possible to switch to EM from any COLREGs state, as opposed to what is implemented in [Eriksen et al., 2020],

For notational brevity, the switching between states are written as

$$q^i(t) = \sigma(\boldsymbol{x}(t),\, \boldsymbol{x}_o^i(t),\, q^i(t^-)), \tag{13}$$

where σ is the switching function[2], while $\boldsymbol{x}(t)$ and $\boldsymbol{x}_o^i(t)$ are the states of the ship and dynamic obstacle i, respectively. The notation $q^i(t^-)$ is used for left-hand limit of q^i at time t.

One commonly used approach for COLREGs-compliant collision avoidance is to assign a spatial region for each obstacle that should not be entered by the ship. The definitions of these spatial regions are not covered by the rules in COLREGs, and there exist several different implementation approaches to obtain the desired behavior. Some examples are to use cost functions [Eriksen et al., 2020, Hagen et al., 2018], forbidden regions [Tam and Bucknall, 2010, Campbell et al., 2014], or velocity obstacles [Kuwata et al., 2013, Kufoalor et al., 2018]. This work uses cost functions, but the method presented in Section 4 can be extended to account for any of the above mentioned approaches.

The cost functions are all based on the relative position of the ship with respect to dynamic obstacles. The relative position between a set $\mathcal{S}_1 \subset \mathbf{R}^2$ and a position $\boldsymbol{y} \in \mathbf{R}^2$ is defined as

$$\text{rel_pos}(\mathcal{S}_1, \boldsymbol{y}) = \underset{\Delta\boldsymbol{p}}{\arg\min}\{\|\Delta\boldsymbol{p}\| \;:\; \boldsymbol{x}-\Delta\boldsymbol{p}=\boldsymbol{y},\, \boldsymbol{x}\in\mathcal{S}_1\}. \tag{14}$$

The COLREGs related cost function is defined as

$$\ell_{\text{CA}}(\boldsymbol{x},\boldsymbol{x}_o^i,q^i) = \begin{cases} k_{q^i} + \ell_{q^i}(\Delta\boldsymbol{p}_o^i,\boldsymbol{x}_o^i) & \Delta\boldsymbol{p}_o^i \in \mathcal{S}_{\text{CA}}(q^i,\boldsymbol{x}_o^i) \\ k_{q^i} & \text{otherwise} \end{cases} \tag{15}$$

where $\Delta\boldsymbol{p}_o^i = \text{rel_pos}(\mathcal{S}_b(\boldsymbol{x}), \boldsymbol{p}_o^i)$ is the relative position of the ship with respect to obstacle i. Moreover, $\ell_{q^i} > 0$ is the cost function related to COLREGs state q^i at state $\boldsymbol{x}_o^i$ and $\mathcal{S}_{\text{CA}}(q^i,\boldsymbol{x}_o^i) \subset \mathbf{R}^2$ is the set of relative positions for which the cost function is active. Finally, k_{q^i} is added such that it is possible to penalize the time spent in the state q^i.

3.2 Problem formulation

The COLREGs-compliant motion planning problem (CCMP) is now defined as follows: compute a feasible and collision-free state and control input trajectory $(\boldsymbol{x}(t),\boldsymbol{u}(t))$, $t\in[0,T_f]$, that moves the ship from its initial state $\boldsymbol{x}(0)=\boldsymbol{x}_0\in\mathcal{X}_{\text{free}}$ to a desired goal state $\boldsymbol{x}(T_f)=\boldsymbol{x}_f\in\mathcal{X}_{\text{free}}$, while minimizing an objective functional J. The cost function used to define J is separated into three parts. The first part is $\ell(\boldsymbol{x},\boldsymbol{u})$, which can, e.g., be related to time and energy consumption. The second part is given by $\ell_s(\boldsymbol{x})$ and penalizes the distance to static obstacles. Finally, the third part is defined in (15) and ensures that the COLREGs rules are not violated for the N_o dynamic obstacles. The CCMP can be posed as the following continuous-time hybrid OCP:

$$\begin{aligned} \underset{\boldsymbol{u}(\cdot),t_f}{\text{minimize}} \quad & \int_{t_0}^{t_f}\left[\ell(\boldsymbol{x},\boldsymbol{u})+\ell_s(\boldsymbol{x}))+\sum_{i=1}^{N_o}\ell_{\text{CA}}(\boldsymbol{x},\boldsymbol{x}_o^i,q^i))\right]\mathrm{d}t \\ \text{subject to} \quad & \boldsymbol{x}(t_0)=\boldsymbol{x}_0, \quad \boldsymbol{x}(t_f)=\boldsymbol{x}_f, \quad q^i(t_0)=q^i(t_0^-) \\ & \dot{\boldsymbol{x}}(t)=\boldsymbol{f}(\boldsymbol{x}(t),\boldsymbol{u}(t)), \\ & q^i(t)=\sigma(\boldsymbol{x}(t),\boldsymbol{x}_o^i(t),q^i(t^-)), \quad i=1\ldots N_o \\ & \boldsymbol{x}(t)\in\mathcal{X}_{\text{free}}(t),\; \boldsymbol{u}(t)\in\mathcal{U}. \end{aligned} \tag{16}$$

The upcoming section describes how the proposed framework for solving (16) is implemented by using a combination of a lattice-based motion planner and an optimization-based improvement step.

4 Optimization-based motion planner

In this section, the two steps of the proposed motion planner are presented in detail, and an algorithm for COLREGs-compliant motion planning is outlined.

4.1 Lattice-based motion planner

A lattice-based motion planner computes a suboptimal solution to the continuous-time hybrid OCP in (16). It is done by discretizing and transforming (16) to a graph-search problem which is solved using suitable techniques. The transformation is done by restricting the control-inputs to a finite subset of the available actions. In this work, the so-called state-lattice methodology is used [Pivtoraiko et al., 2009], where the search space is discretized into $\mathcal{X}_d$ and the set of available actions is represented using a motion-primitive set $\mathcal{P}$.

A motion primitive $\boldsymbol{m}$ is defined as a feasible state and control-input trajectory that moves the ship from an initial state $\boldsymbol{x}_0\in\mathcal{X}_d$ to a terminal state $\boldsymbol{x}_f\in\mathcal{X}_d$ in time T. By discretizing the time uniformly at $t_j = 0+j\Delta_{t_m}$, $j=0,\ldots N_m$, a sampled version of a motion primitive at these time instances is defined as

$$\boldsymbol{m} = \{\boldsymbol{x}_0,\, \boldsymbol{u}_0,\, \boldsymbol{x}_1,\boldsymbol{u}_1,\ldots,\boldsymbol{x}_{N_m}\}. \tag{17}$$

To compute the motion primitives $\boldsymbol{m}\in\mathcal{P}$, the optimization-based framework in [Bergman et al., June 2019] is used. Since the positions of the obstacles are unknown offline, the motion primitives are computed by solving OCPs where only a discretized version of the position invariant cost

$$\ell_{p,m}(\boldsymbol{m}) = \Delta_{t_m}\sum_{j=0}^{N_m-1}\ell(\boldsymbol{x}_j,\boldsymbol{u}_j) \tag{18}$$

[2]The switching function is allowed to depend on the type of dynamic obstacle.

is minimized. More details on the motion primitive computation for the ship is found in [Bergman et al., 2020b]. Together, the motion primitives define the motion primitive set $\mathcal{P}$. Since the ship is position invariant, each motion primitive $\boldsymbol{m} \in \mathcal{P}$ can be translated and reused from each position in $\mathcal{X}_d$. The state transition function is defined as $\boldsymbol{x}_j = \boldsymbol{f}_m(\bar{\boldsymbol{x}}, \boldsymbol{m}, t_j)$ which outputs the state after t_j seconds when motion primitive $\boldsymbol{m}$ is applied from $\bar{\boldsymbol{x}}$.

The obstacle dependent cost functions are evaluated online during the graph search. When a motion primitive $\boldsymbol{m}$ is applied from state $\bar{\boldsymbol{x}}$, the cost related to static obstacles is

$$\ell_{s,m}(\bar{\boldsymbol{x}}, \boldsymbol{m}) = \Delta_{t_m} \sum_{j=0}^{N_m-1} \ell_s(\boldsymbol{x}_j), \tag{19}$$

where $\boldsymbol{x}_j = \boldsymbol{f}_m(\bar{\boldsymbol{x}}, \boldsymbol{m}, t_j)$. If the location of the static obstacles are known offline, it is possible to precompute an approximation of this function in terms of a static cost map. For the dynamic obstacles, it is assumed that a prediction of the state of the obstacle $\boldsymbol{x}_o^i(\cdot)$ is given, such that the COLREGs state trajectory $\boldsymbol{q}^i = \{q_0^i, \dots q_{N_m-1}^i\}$ can be computed for a motion primitive $\boldsymbol{m}$ applied at state $\bar{\boldsymbol{x}}$. This is used to define the cost related to dynamic obstacles for $\boldsymbol{m}$ applied at $\bar{\boldsymbol{x}}$:

$$\ell_{\mathrm{CA},m}\left(\bar{\boldsymbol{x}}, \boldsymbol{m}, \boldsymbol{x}_o^i(\cdot), \boldsymbol{q}^i\right) = \Delta_{t_m} \sum_{j=0}^{N_m-1} \ell_{\mathrm{CA}}\left(\boldsymbol{x}_j, \boldsymbol{x}_o^i(\bar{t}_0 + t_j), q_j^i\right), \tag{20}$$

where $\boldsymbol{x}_j = \boldsymbol{f}_m(\bar{\boldsymbol{x}}, \boldsymbol{m}, t_j)$ and $\bar{t}_0$ is the time when the motion primitive is applied. One approach to approximate this function online is to compute the relative position to $\boldsymbol{x}_o^i(\bar{t}_0 + t_j)$ for a finite number of vertices in the ownship's body $\mathcal{S}_b(\boldsymbol{x}_j)$.

By summarizing the terms in (18)–(20), the total stage cost when a motion primitive is applied is

$$\begin{aligned} &\ell_m\left(\bar{\boldsymbol{x}}, \boldsymbol{m}, \{\boldsymbol{x}_o^i(\cdot)\}_{i=1}^{N_o}, \{\boldsymbol{q}^i\}_{i=1}^{N_o}\right) = \\ &\ell_{p,m}(\boldsymbol{m}) + \ell_{s,m}(\bar{\boldsymbol{x}}, \boldsymbol{m}) + \sum_{i=1}^{N_o} \ell_{\mathrm{CA},m}\left(\bar{\boldsymbol{x}}, \boldsymbol{m}, \boldsymbol{x}_o^i(\cdot), \boldsymbol{q}^i\right). \end{aligned} \tag{21}$$

Now, the resulting graph-search problem can be posed as a discrete OCP in the form:

$$\begin{aligned} \underset{\{\boldsymbol{m}_k\}_{k=1}^M,\, M}{\text{minimize}} \quad & \sum_{k=1}^{M} \ell_m\left(\bar{\boldsymbol{x}}_k, \boldsymbol{m}_k, \{\boldsymbol{x}_o^i(\cdot)\}_{i=1}^{N_o}, \{\bar{\boldsymbol{q}}_k^i\}_{i=1}^{N_o}\right) \\ \text{subject to} \quad & \bar{\boldsymbol{x}}_1 = \boldsymbol{x}_0, \quad \bar{\boldsymbol{x}}_{M+1} = \boldsymbol{x}_f, \quad \bar{\boldsymbol{q}}_0^i = q_i(t_0^-) && \text{(22a)} \\ & \bar{\boldsymbol{x}}_{k+1} = \boldsymbol{f}_m(\bar{\boldsymbol{x}}_k, \boldsymbol{m}_k, t_{N_{m_k}}), && \text{(22b)} \\ & \bar{\boldsymbol{q}}_k^i = \boldsymbol{\sigma}_m(\bar{\boldsymbol{x}}_k, \boldsymbol{m}_k, \boldsymbol{x}_o^i(\cdot), \bar{\boldsymbol{q}}_{k-1}^i) && \text{(22c)} \\ & \boldsymbol{m}_k \in \mathcal{P}(\boldsymbol{x}_k), && \text{(22d)} \\ & c(\bar{\boldsymbol{x}}_k, \boldsymbol{m}_k) \notin \mathcal{X}_{\mathrm{obst}}, && \text{(22e)} \end{aligned}$$

where the decision variables are the motion-primitive sequence $\{\boldsymbol{m}_k\}_{k=1}^M$ and its length M. The function in (22c) outputs the COLREGs state trajectory for dynamic obstacle i along $\boldsymbol{m}_k$ applied from $\bar{\boldsymbol{x}}_k$, and (22d) restricts the choice of $\boldsymbol{m}_k$ to the set of applicable motion primitives $\mathcal{P}(\bar{\boldsymbol{x}}_k)$ from state $\bar{\boldsymbol{x}}_k$. Finally, the constraint in (22e) ensures that the ship does not collide with obstacles when $\boldsymbol{m}_k$ is applied from $\bar{\boldsymbol{x}}_k$.

Since the problem in (22) is in the form of a graph-search problem, standard graph-search algorithms can be used to solve it [Pivtoraiko et al., 2009]. For a faster search, a precomputed free-space heuristic look-up table (HLUT) [Knepper and Kelly, 2006] is used as heuristic function. The output is a state and control-input trajectory $(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$ which is defined by the motion primitive sequence. Additional outputs are the COLREGs state trajectories for all obstacles. For dynamic obstacle i, it is defined by

$$\bar{q}^i(t) = q_k^i \quad \underline{t}_k \le t < \bar{t}_k, \quad k = 1, \dots, N_k \tag{23}$$

where $\underline{t}_k$ and $\bar{t}_k$ with $\underline{t}_k = \bar{t}_{k-1}$ define the time interval where COLREGs state q_k^i is active, and N_k is the number of COLREGs state switches.

4.2 Receding-horizon improvement

The aim of the receding-horizon improvement step is to reduce the suboptimality introduced from solving (22). This is achieved by performing local improvements to the trajectories from the lattice-based planner $(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$ in a receding-horizon fashion over a user-defined planning horizon T. This step is based on the work in [Bergman et al., 2020b], and is here extended to account for dynamic obstacles and COLREGs.

By using direct methods for optimal control, the improvement OCP is constructed by discretizing the continuous-time motion planning problem (16), e.g., using direct multiple-shooting with $N+1$ discretization points [Diehl et al., 2006], where the time between two points is $\Delta_t = T/N$. To avoid collisions with both static and dynamic obstacles, a local spatial constraint is computed at each discrete point j, $j = 1, \dots, N-1$. This constraint is represented by a convex polytope $\mathcal{S}_{\mathrm{env}}^j$ which is defined as

$$\mathcal{S}_{\mathrm{env}}^j = \{\boldsymbol{p} \in \mathbf{R}^2 \mid \boldsymbol{A}_j \boldsymbol{p} \le \boldsymbol{b}_j\}, \tag{24}$$

where $\boldsymbol{A}_j = [\boldsymbol{a}_{j,1} \ \dots \ \boldsymbol{a}_{j,K_j}]^\intercal \in \mathbf{R}^{K_j \times 2}$, $\boldsymbol{a}_{j,l} \in \mathbf{R}^2$, $||\boldsymbol{a}_{j,l}||_2 = 1$, for $l = 1, \dots, K_j$, $\boldsymbol{b}_j \in \mathbf{R}^{K_j}$ and K_j is the number of half spaces that defines $\mathcal{S}_{\mathrm{env}}^j$. These polytopes are computed using Algorithm 1 in [Bergman et al., 2020b]. Since both $\mathcal{S}_b(\boldsymbol{x}_j)$ and $\mathcal{S}_{\mathrm{env}}^j$ are convex, a sufficient condition to ensure collision avoidance for the ship's body is that all vertices $\mathcal{V}_b$ of $\mathcal{S}_b(\boldsymbol{x}_j)$ lie within $\mathcal{S}_{\mathrm{env}}^j$ [Martinsen et al., 2019]. This condition can be described by the following constraint:

$$\boldsymbol{A}_j \boldsymbol{c}_{\mathrm{rot}}(\boldsymbol{x}_j, \boldsymbol{p}_l) \le \boldsymbol{b}_j, \quad \forall \boldsymbol{p}_l \in \mathcal{V}_b, \tag{25}$$

where

$$\boldsymbol{c}_{\mathrm{rot}}(\boldsymbol{x}_j, \boldsymbol{p}_l) = \begin{bmatrix} \cos\psi_j & -\sin\psi_j \\ \sin\psi_j & \cos\psi_j \end{bmatrix} \boldsymbol{p}_l + \begin{bmatrix} x_j \\ y_j \end{bmatrix}, \tag{26}$$

and $\boldsymbol{A}_j$ and $\boldsymbol{b}_j$ are the half-space representation of $\mathcal{S}_{\mathrm{env}}^j$ defined in (24). Furthermore, we want to express the position-dependent cost related to the distance to obstacles also in the

improvement step. Here, this is done by modifying (25) to:

$$\boldsymbol{A}_j \boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_j, \boldsymbol{p}_l) \leq \boldsymbol{b}_j - \boldsymbol{1}_j (d_{\text{safe}} - \epsilon_{d,j}), \quad \forall \boldsymbol{p}_l \in \mathcal{V}_b, \tag{27}$$

where a variable $\epsilon_{d,j}$ that satisfies $0 \leq \epsilon_{d,j} \leq d_{\text{safe}}$ is added, which represents the distance to the boundaries of $\mathcal{S}^j_{\text{env}}$ up to a user-defined safety distance d_{safe}. With this modification, the objective function in the improvement step is

$$\begin{aligned} &\ell_{\text{imp}}\left(\boldsymbol{x}, \boldsymbol{u}, \epsilon_d, \{\boldsymbol{x}^i_o, q^i\}_{i=1}^{N_o}\right) = \\ &\ell(\boldsymbol{x}, \boldsymbol{u}) + k_d \epsilon_d^2 + \sum_{i=1}^{N_o} \ell_{\text{CA}}(\boldsymbol{x}, \boldsymbol{x}^i_o . q^i). \end{aligned} \tag{28}$$

From the current state $\boldsymbol{x}_{\text{cur}}$ at time t_{cur}, the receding-horizon improvement OCP is

$$\begin{aligned} \underset{\{\boldsymbol{u}_j, \epsilon_{d,j}\}_{j=0}^{N-1}}{\text{minimize}} \quad & \Delta_t \sum_{j=0}^{N-1} \left[\ell_{\text{imp}}(\boldsymbol{x}_j, \boldsymbol{u}_j, \epsilon_{d,j}, \{\boldsymbol{x}^i_o(t_j), \bar{q}^i(t_j)\}_{i=1}^{N_o})\right] \\ \text{subject to} \quad & \boldsymbol{x}_0 = \boldsymbol{x}_{\text{cur}}, \quad \boldsymbol{x}_N = \bar{\boldsymbol{x}}(t_{\text{cur}} + T), \quad t_j = t_{\text{cur}} + j\Delta_t, \\ & \boldsymbol{x}_{j+1} = \boldsymbol{f}_d(\boldsymbol{x}_j, \boldsymbol{u}_j, \Delta_t), \\ & \boldsymbol{A}_j \boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_j, \boldsymbol{p}_l) \leq \boldsymbol{b}_j - \boldsymbol{1}_j (d_{\text{safe}} - \epsilon_{d,j}), \forall \boldsymbol{p}_l \in \mathcal{V}_b \\ & \boldsymbol{x}_j \in \mathcal{X}, \ \boldsymbol{u}_j \in \mathcal{U}, \quad 0 \leq \epsilon_{d,j} \leq d_{\text{safe}}. \end{aligned} \tag{29}$$

Here, $\boldsymbol{f}_d(\boldsymbol{x}, \boldsymbol{u}, \Delta_t)$ is a discretized version of (4) over the time interval Δ_t. Furthermore, the terminal state $\boldsymbol{x}_N$ is constrained such that the solution is connected to the initial trajectory $\bar{\boldsymbol{x}}(\cdot)$ at the end of the planning horizon, which is done for stability guarantees [Bergman et al., 2020a]. Compared to the initial problem formulation (16), the combinatorial aspect of selecting the COLREGs state trajectories $\bar{q}^i(\cdot)$ is in the improvement step pre-determined by the lattice-based planner as defined in (23). Furthermore, the second combinatorial aspect of selecting on which side to pass obstacles is implicitly encoded in the warm-start trajectory from the lattice planner. Hence, the hybrid OCP in (16) has been transformed into a standard discrete-time OCP, which can be solved using direct OCP techniques such as the ones presented in [Diehl et al., 2006].

4.3 Proposed CCMP algorithm

In an online setting, an update in the predicted trajectories of the dynamic obstacles might require a re-planning. To handle such scenarios, a CCMP algorithm is proposed in Algorithm 1 that is based on the two steps described in Section 4.1 and Section 4.2.

In the first step on line 2, the lattice-based motion planner is used to compute the initial state and control-input trajectories $(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$ and COLREGs state trajectories $\{\bar{q}^i(\cdot)\}_{i=1}^{N_o}$ by solving (22). Then, $(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$ is improved in a receding-horizon fashion until the terminal state is reached. Due to uncertainties in the prediction of the dynamic obstacle trajectories $\{\boldsymbol{x}^i_o(\cdot)\}_{i=0}^{N_o}$, and that new dynamic obstacles might appear during execution, the predictions are updated at the beginning of each improvement iteration (line 5). Then, the associated COLREGs state trajectories are updated on line 6 according

Algorithm 1 Proposed CCMP algorithm

1: **input**: $\boldsymbol{x}_0$, $\boldsymbol{x}_f$, $\mathcal{P}$, $\mathcal{X}^s_{\text{obst}}$, $\{\boldsymbol{x}^i_o(\cdot)\}_{i=1}^{N_o}$, T, Δ_t
2: $\left(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot), \{\bar{q}^i(\cdot)\}_{i=1}^{N_o}\right) \leftarrow$ Solve (22) with $\mathcal{P}$
3: $t_{\text{cur}} \leftarrow t_0 + \Delta_t$
4: **while** $\bar{\boldsymbol{x}}(t_{\text{cur}} + T) \neq \boldsymbol{x}_f$ **do**
5: $\quad \{\boldsymbol{x}^i_o(\cdot)\}_{i=1}^{N_o} \leftarrow$ `obstacles_prediction()`
6: $\quad \bar{q}^i(t) \leftarrow \sigma\left(\bar{\boldsymbol{x}}(t), \bar{\boldsymbol{x}}^i_o(t), \bar{q}^i(t^-)\right)$, $t_{\text{cur}} \leq t \leq t_f$
7: $\quad$ **if any** $\bar{q}^i(t_{\text{cur}}) = \text{EM}$ **then**
8: $\quad\quad$ `emergency_maneuver`$(\bar{\boldsymbol{x}}(t_{\text{cur}}))$
9: $\quad$ **else if any** $\bar{\boldsymbol{x}}(t) \in \mathcal{X}^d_{\text{obst}}(t)$, $t \geq t_{\text{cur}} + T$ **then**
10: $\quad\quad \left(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot), \{\bar{q}^i(\cdot)\}_{i=1}^{N_o}\right) \leftarrow$ Solve (22) from $\bar{\boldsymbol{x}}(t_{\text{cur}})$
11: $\quad$ **else**
12: $\quad\quad \{\boldsymbol{A}_j, \boldsymbol{b}_j\}_{j=1}^{N-1} \leftarrow$ `prepare_improvement`$(\bar{\boldsymbol{x}}(\cdot))$
13: $\quad\quad (\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot)) \leftarrow$ Solve (29) from $\boldsymbol{x}_{\text{cur}} = \bar{\boldsymbol{x}}(t_{\text{cur}})$
14: $\quad\quad$ re_plan $\leftarrow$ `check_trajectory`$(\bar{\boldsymbol{x}}(\cdot))$
15: $\quad\quad$ **if** re_plan **is** True **then**
16: $\quad\quad\quad \left(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot), \{\bar{q}^i(\cdot)\}_{i=1}^{N_o}\right) \leftarrow$ Solve (22) from $\bar{\boldsymbol{x}}(t_{\text{cur}})$
17: $\quad\quad$ **end if**
18: $\quad$ **end if**
19: $\quad$ Send trajectory to controller : `send_reference`$((\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$
20: $\quad t_{\text{cur}} \leftarrow t_{\text{cur}} + \Delta_t$
21: **end while**

to the new obstacle predictions. If any of the COLREGs state trajectories are in state EM, an emergency maneuver from the current state is computed. There are several ways to compute such a maneuver, e.g., to use the lattice-based motion planner for computing a safe stop maneuver based on the current situation.

If the updated dynamic obstacle predictions cause a collision beyond the current improvement planning horizon $t_{\text{cur}}+T$ (line 9), a new trajectory is computed by the lattice-based planner. Otherwise, a preparation step is performed where the initial and terminal constraints are specified, the objective function is parameterized based on $\bar{q}^i(t)$ and the collision avoidance polytopes $\boldsymbol{A}_j, \boldsymbol{b}_j$ are computed. After the preparation step, (29) is solved from $\boldsymbol{x}_{\text{cur}}$ on line 13. If the trajectory from solving (29) triggers re-planning, e.g., if it is close to infeasibility or the objective function value has increased due to that the dynamic obstacle predictions have changed, the lattice-based motion planner computes a new trajectory from $\boldsymbol{x}_0 = \bar{\boldsymbol{x}}(t_{\text{cur}})$. Finally, the state and control-input trajectories are sent on line 19 to a trajectory-tracking controller such as [Bärlund et al., 2020] to stabilize the ship and suppress various disturbances.

5 Simulation study

In this section, the proposed framework is evaluated in a simulation study by solving motion planning problems in a model of the Helsinki archipelago[3]. The geometry of the environ-

[3] Video available at `https://youtu.be/yPEFJRywWHQ`

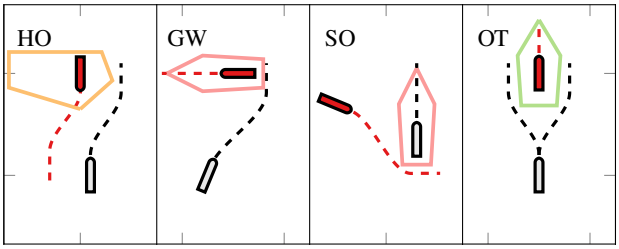


Figure 4: Different polygons used to represent the restricted regions associated with the COLREGs scenarios shown in Fig. 2.

ment is represented using polygons that have been calculated using data from Open Street Map and the Finnish Transport and Communications Agency (Traficom). These polygons have been expanded using depth data such that only navigable water is represented, see, e.g., Fig. 5. The cost map used in the lattice-based planner is computed with resolution 1 m. The motion planner is implemented in C++, where the OCPs are solved using CasADi [Andersson et al., In Press, 2018] with IPOPT [Wächter and Biegler, 2006] as NLP solver, and discretized using multiple shooting. From the results in [Bergman et al., 2020b], it was found that a planning horizon of $T = 150$ s with $\Delta_t = 2$ s are suitable values to use for the supply vessel with respect to the trade-off between computation time and optimality. These values are used in all scenarios presented in this section when solving the improvement OCP in (29), which means that the improvement step runs at a frequency of 0.5 Hz.

The simulations are based on a model of a small ship with two thrusters. The model of the ship is derived from the *supply vessel* model in the MSS hydro toolbox [Perez et al., 2006]. The parameters for the two thrusters are:

$$\bar{n} = 2 \text{ RPS} \quad \bar{\dot{n}} = 0.08 \text{ RPS/s}, \quad \bar{\dot{\alpha}} = 7.2^\circ/\text{s}, \tag{30}$$

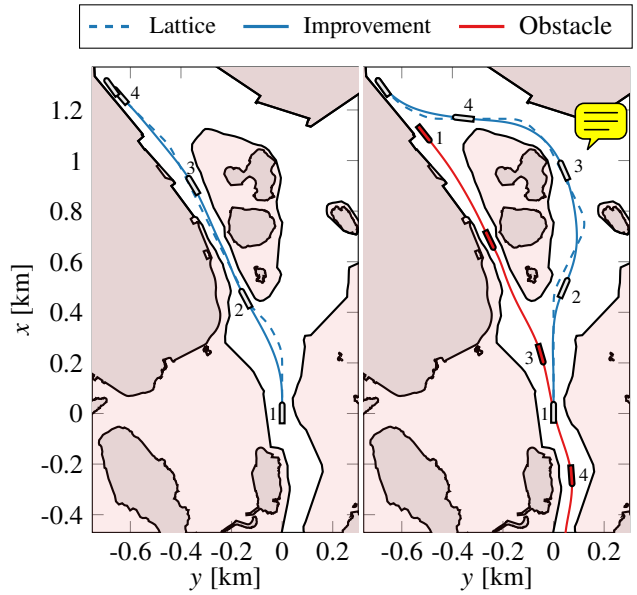
The lattice-based planner is discretized with a position resolution of $r_p = 5$ m, and the maximum velocity is set to 6 kn. More details of the model parameters and the motion primitives used in the lattice-based planner are found in [Bergman et al., 2020b].

The position-invariant cost function used in both steps of the framework is selected as

$$\ell(\boldsymbol{x}, \boldsymbol{u}) = 1 + 0.1\boldsymbol{n}^\top \boldsymbol{n} + 100\dot{\boldsymbol{n}}^\top \dot{\boldsymbol{n}} + 100\dot{\boldsymbol{\alpha}}^\top \dot{\boldsymbol{\alpha}}, \tag{31}$$

to trade-off between time, energy and smoothness. As for the cost related to the distance to obstacles, the value of the weighting parameter is selected as $k_d = 1.5 \cdot 10^{-3}$ in (21) and (28). Furthermore, the safety distance parameter is set to $d_{\text{safe}} = 20$ m which is approximately equal to the width of the ship's body. Finally, the COLREGs-related cost function is implemented as hard constraints, which corresponds to an infinite cost in (21) and (28). The regions for each COLREGs situation are represented as polygons and are illustrated in Fig. 4[4]. The threshold parameters used to switch between

[4]The current implementation uses a fixed size of the polygons, but can be extended to depend on the velocity of the dynamic obstacle as in [Tam and Bucknall, 2010].



(a) No dynamic obstacle (P1) (b) Dynamic obstacle (P2)

Figure 5: Two similar docking scenarios where the white area represents navigable water. The vessels are plotted at given timestamps (1-4) for interpretabilty. In (a), the motion planner finds a trajectory through the narrow passage. In (b), a dynamic obstacle occupies the narrow passage, which makes the proposed motion planner to find a trajectory on the other side of the island such that it completely avoids a complex COLREGs situation.

COLREGs states in (10)-(11) are

$$\begin{aligned} \underline{d} &= 200, \ \underline{t}_{\text{low}} = -10, \ \underline{t}_{\text{high}} = 150, \\ \bar{d} &= 240, \ \bar{t}_{\text{low}} = -25, \ \bar{t}_{\text{high}} = 200. \end{aligned} \tag{32}$$

The motion-planning scenarios are illustrated in Fig. 5–8, where four different trajectories are computed to different docking states with zero velocity. Since the performance of the proposed motion planner is evaluated in this section, the simulations are performed without any disturbances and the open-loop control law is used for evaluation. Moreover, the ship is aware of the future dynamic obstacle trajectories up to 150 seconds. After that, constant-velocity predictions are used for another 650 seconds.

The result in Fig. 5a shows that the proposed motion planner is able to compute trajectories through narrow passages. In Fig. 5b, it can be seen that the motion planner is able to compute a trajectory that completely avoids a difficult COLREGs situation when another vessel is occupying the passage, by passing on the other side of the island. This is in contrast to methods that follows an initial reference computed without considering dynamic obstacles, such as [Eriksen et al., 2020], which instead react after the COLREGs situation has already occurred and hence is forced to wait until the other vessel has passed.

In Fig. 6, a crossing scenario (P3) is illustrated where the dynamic obstacle is approaching from the starboard side. This is identified correctly by the motion planner which switches to state GW and yields to the dynamic obstacle. It can be

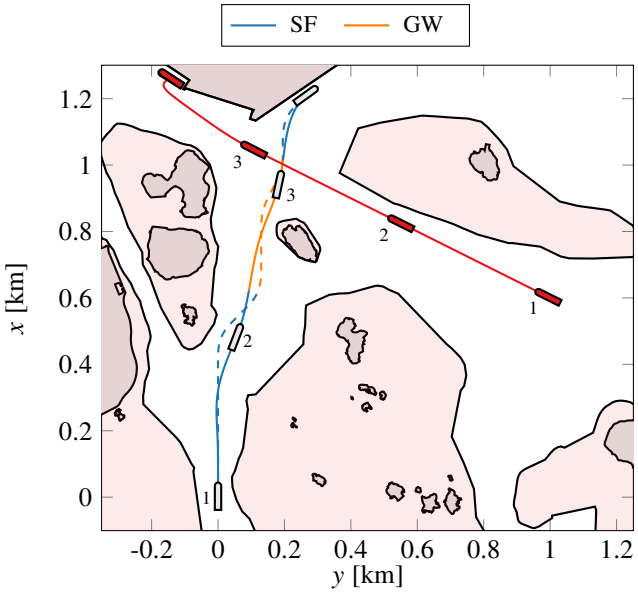


Figure 6: (P3) A crossing scenario where the dynamic obstacle is approaching from the starboard side. The vessels are plotted at given timestamps (1-3) for interpretabilty. Path segments in orange represent parts where the ship is in state GW.

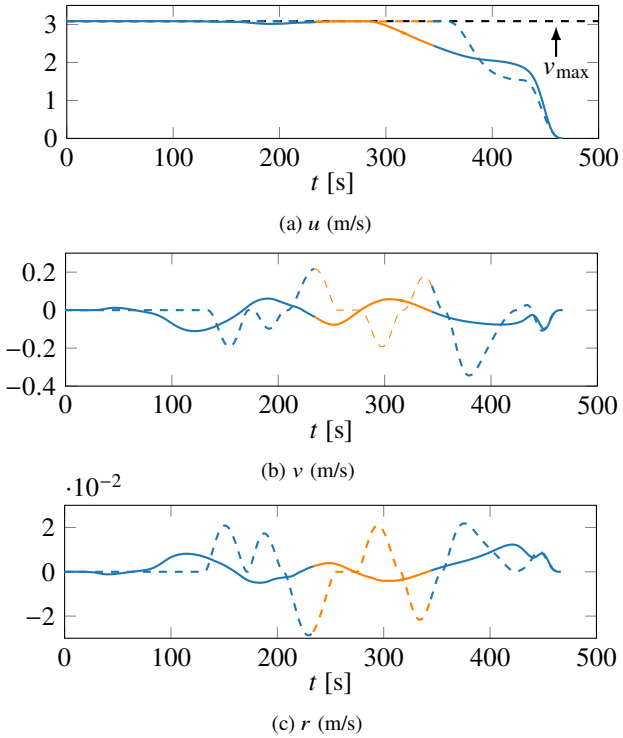


Figure 7: Linear velocity trajectories for the ship during the crossing scenario in Fig. 6. Trajectory segments in orange represent parts where the ship is in state GW.

seen that the trajectory from the lattice-based motion planner is oscillatory. This behavior is explained by the following observations:

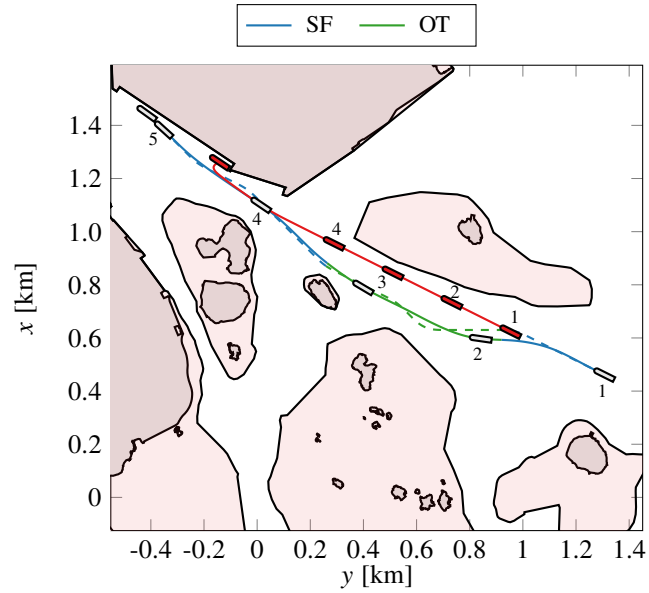


Figure 8: (P4) A scenario where the ship is able to overtake the dynamic obstacle. The vessels are plotted at given timestamps (1-5) for interpretabilty. Path segments in green represent parts where the ship is in state OT.

1. The small island prevents the planner from performing a course change to pass behind the dynamic obstacle.
2. The lattice-based planner is limited to select motion primitives with velocities in $\mathcal{X}_d$, making it more beneficial to oscillate than to slow down.

The oscillatory behavior is then reduced by the improvement step, which can use all feasible velocities and is thus able to slow down instead of oscillating as seen in Fig. 7.

The last scenario (P4) is shown in Fig. 8, where the motion planner finds a trajectory that safely overtakes the dynamic obstacle which is moving at a speed of 3 kn. A summary of the results from all scenarios is given in Table 1. One notable result is the increase in computation time for the lattice-based planner for problem P2. An explanation to this increase is that the heuristic estimate in the HLUT is far from correct since it is unaware of the dynamic obstacle blocking the passage. Another result is that the improvement step is able to significantly reduce the aggregated applied force by the thrusters in all scenarios, and thus provide more smooth and energy-efficient trajectories compared to only using the lattice-based planner.

6 Conclusions and Future Work

This work proposes a two-step optimization-based motion planner for marine vessels that complies with the international regulations for preventing collisions at sea (COLREGs). It extends the work presented in [Bergman et al., 2020b] such that the motion planner is able to account for dynamic obstacles and the rules relevant for trajectory planning in COLREGs. In

Table 1: Results for the motion planning problems in Fig. 5–8. The variable t_{lat} represents the time for the lattice-based planner to compute the initial trajectory, and t_f is the time it takes to execute the trajectory. Finally, F_f is the aggregated applied force for the initial lattice-based trajectory (lat) and after the improvement step (imp).

Problem	t_{lat} [s]	t_f [s]	$F_{f,\text{lat}}$ [MN]	$F_{f,\text{imp}}$ [MN]
P1	0.11	533	71.3	52.7
P2	10.2	645	93.4	67.3
P3	0.24	467	67.8	46.4
P4	0.18	711	97.5	69.3

the first step, a lattice-based motion planner computes a sub-optimal trajectory based on a finite, precomputed library of motion primitives. This step is here augmented with discrete states that represent the COLREGs situations of the ship with respect to other vessels. In the second step, direct optimal control techniques are applied to locally improve the trajectory from the lattice-based planner while keeping the COLREGs states fixed. The proposed motion planner is evaluated in a model of the Helsinki archipelago, where it successfully computes trajectories that complies with COLREGs. The results also show the motion planner's capability to completely avoid complex collision situations from occurring at all.

Future work includes to develop a heuristic function in the lattice-based planner that also captures both static and dynamic obstacles, to develop and evaluate the robustness of the implementation when the future behavior of the dynamic obstacles is uncertain, and to implement an emergency strategy.

References

J. A. E. Andersson et al. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, In Press, 2018.

A. Bärlund et al. Nonlinear MPC for combined motion control and thrust allocation of ships. In *21st IFAC World Congress*, 2020.

K. Bergman, O. Ljungqvist, T. Glad, and D. Axehill. An optimization-based receding horizon trajectory planning algorithm. In *21st IFAC World Congress*, 2020a.

K. Bergman, O. Ljunqvist, J. Linder, and D. Axehill. An optimization-based motion planner for autonomous maneuvering of marine vessels in complex environments. In *59th IEEE Conference on Decision and Control (CDC)*, 2020b.

K. Bergman, O. Ljungqvist, and D. Axehill. Improved optimization of motion primitives for motion planning in state lattices. In *2019 IEEE Intelligent Vehicles Symposium (IV)*, June 2019.

G. Bitar, A. B. Martinsen, A. M. Lekkas, and M. Breivik. Two-stage optimized trajectory planning for ASVs under polygonal obstacle constraints: Theory and experiments. *IEEE Access*, 8:199953–199969, 2020.

S. Campbell, M. Abu-Tair, and W. Naeem. An automatic COLREGs-compliant obstacle avoidance system for an unmanned surface vehicle. *Proceedings of the Institution of Mechanical Engineers, Part M: Journal of Engineering for the Maritime Environment*, 228(2):108–121, 2014.

H.-T. L. Chiang and L. Tapia. COLREG-RRT: An RRT-based COLREGS-compliant motion planner for surface vehicle navigation. *IEEE Robotics and Automation Letters*, 3(3):2024–2031, 2018.

A. N. Cockcroft and J. N. F. Lameijer. *Guide to the collision avoidance rules*. Elsevier, 2003.

M. Diehl et al. Fast direct multiple shooting algorithms for optimal robot control. In *Fast motions in biomechanics and robotics*, pages 65–93. Springer, 2006.

B.-O. H. Eriksen, G. Bitar, M. Breivik, and A. M. Lekkas. Hybrid collision avoidance for ASVs compliant with COLREGs rules 8 and 13–17. *Frontiers in Robotics and AI*, 7:11, 2020.

T. I. Fossen. *Handbook of marine craft hydrodynamics and motion control*. John Wiley & Sons, 2011.

I. B. Hagen, D. K. M. Kufoalor, E. F. Brekke, and T. A. Johansen. MPC-based collision avoidance strategy for existing marine vessel guidance systems. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7618–7623. IEEE, 2018.

International Maritime Organization. IMO takes first steps to address autonomous ships. accessed: Dec. 21, 2020, [online], available: https://www.imo.org/en/MediaCentre/PressBriefings/Pages/08-MSC-99-MASS-s 2020.

R. A. Knepper and A. Kelly. High performance state lattice planning using heuristic look-up tables. In *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3375–3380, 2006.

D. K. M. Kufoalor, E. F. Brekke, and T. A. Johansen. Proactive collision avoidance for ASVs using a dynamic reciprocal velocity obstacles method. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2402–2409. IEEE, 2018.

Y. Kuwata, M. T. Wolf, D. Zarzhitsky, and T. L. Huntsberger. Safe maritime autonomous navigation with COLREGS, using velocity obstacles. *IEEE Journal of Oceanic Engineering*, 39(1):110–119, 2013.

S. M. LaValle. *Planning Algorithms*. Cambridge University Press, Cambridge, UK, 2006.

A. B. Martinsen, A. M. Lekkas, and S. Gros. Autonomous docking using direct optimal control. In *Proc. 12th IFAC Conference Control Applications in Marine Systems, Robotics, and Veh. (CAMS)*, volume 52, pages 97–102. IFAC, 2019.

A. B. Martinsen, G. Bitar, A. M. Lekkas, and S. Gros. Optimization-based automatic docking and berthing of ASVs using exteroceptive sensors: Theory and experiments. *IEEE Access*, 8:204974–204986, 2020.

T. Perez. *Ship motion control: course keeping and roll stabilisation using rudder and fins*. Springer Science & Business Media, 2006.

T. Perez et al. An overview of the marine systems simulator (MSS): A simulink toolbox for marine control systems. *Modeling, Identification and Control*, 27(4):259–275, 2006.

M. Pivtoraiko, R. A. Knepper, and A. Kelly. Differentially constrained mobile robot motion planning in state lattices. *Journal of Field Robotics*, 26(3):308–333, 2009.

C. Tam and R. Bucknall. Collision risk assessment for ships. *Journal of marine science and technology*, 15(3):257–270, 2010.

A. Wächter and L. T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106(1):25–57, 2006.

M. Wingrove. Industry sails course towards autonomous shipping. Riviera, accessed: Dec. 21, 2020, [online], available: https://www.rivieramm.com/news-content-hub/news-content-hub/industry-sails-course-towards-autonomous-shipping-60359, 2020.